UNIVERSITY OF HAVANA

Faculty of Mathematics and Computer Science

DATABASES II

Report: Database Design

Management for the Automatic Generation of Delivery Routes

Academic year: 2026–2027
Team: Digimons

Member	Group	Telegram	GitHub
Karla Díaz Saura	C312	@kdiazsr	Krli01
Manuel Enrique Cardoso Martínez	C312	@ManuEC04	ManuEC04
Leonardo Peláez Ascención	C312	@leosoyyoxd	Brand-Leo
Jorge Luis Herrera Cecilia	C312	@Jorge_Lhc	Jorge-Luis-Herrera
José Alejandro Albanés Febles	C311	@cuban_developer	JoseAlejandro0508

Report prepared according to the Methodological Guidelines for Databases II.

Contents

1 Project title **2**

2 Analysis and reformulation of requirements **2**

2.1 General description 2

2.2 Actors and responsibilities 2

2.3 Reformulated functional requirements 2

2.4 Informational requirements 3

2.5 Non-functional requirements 3

3 Proposed technology stack and architecture **4**

3.1 Technology stack 4

3.2 Architecture and communication 5

3.3 Patterns, quality, and operation 5

4 Extended Relational Conceptual Model (ERCM) **5**

4.1 Conceptual view 5

4.2 Main entities and relationships 6

5 Integrity constraints **7**

5.1 Entity integrity 7

5.2 Referential integrity 7

5.3 Business constraints 7

6 Assessment of design correctness **8**

6.1 Functional dependencies and normalization 8

6.2 Dependency preservation and lossless join 8

6.3 Recommended implementation decisions 8

7 Conclusions **9**

Sources used **9**

1 Project title

Management for the Automatic Generation of Delivery Routes.

2 Analysis and reformulation of requirements

2.1 General description

The project starts from a practical need: to bring order to a delivery operation involving orders, centers, zones, coordinators, supervisors, and drivers. The web application will register orders and stops, organize territorial coverage, generate routes, validate plans, execute deliveries, and analyze their results. The database will follow that entire process so that every decision can be traced from order entry through delivery or rescheduling.

The solution combines a relational model for transactional information with an external graph engine for the dynamic network of roads and transit points. This separation enables efficient queries and alternative-route traversal without introducing into the relational schema a structure that changes and is traversed continuously.

2.2 Actors and responsibilities

Actor	Main responsibilities
Administrator	Manage users and general parameters; prevent duplicate orders and incompatible route assignments.
Logistics coordinator	Register and classify orders, operate in authorized centers, select loads, generate routes, and resolve assigned rescheduling requests.
Operations supervisor	Review routes, approve or reject them, request modifications, and record observations.
Driver	Consult assigned routes, execute stops, register deliveries and incidents, and request rescheduling.

2.3 Reformulated functional requirements

1. **Authentication and authorization.** The system will centralize user identity and apply permissions according to four roles: administrator, logistics coordinator, operations supervisor, and driver. The hierarchy is total and disjoint: every operational user belongs to one subtype and does not simultaneously belong to incompatible subtypes.
2. **Centers, zones, and stops.** Distribution centers, their covered zones, and destination stops will be registered. A center may cover several zones, and a zone may be served by several centers.

3. **Order registration.** An authorized coordinator will enter each order, specifying its description, delivery type (parcel, document, or perishable), priority (low, medium, or urgent), origin center, and destination stop.
4. **Automatic route generation.** The system will select orders and generate a route considering vehicle capacity, territorial distribution, delivery types, priority, and balancing criteria. The parameters used will be stored with the route.
5. **Supervision.** Every proposed route may be reviewed by a supervisor. The review result, date, and observations will be preserved for auditing.
6. **Assignment and execution.** An approved route may be assigned to a driver authorized for the corresponding center. The system will prevent the driver from having two incompatible active routes.
7. **Deliveries and rescheduling.** The driver will record the result of each delivery and its observations. In case of failure, the driver may create a rescheduling request and designate the coordinator who will resolve it; the manual result will be preserved.
8. **Reports and export.** All users may consult tabular and graphical results, sort columns, and export information to PDF.

2.4 Informational requirements

The database must support at least the following queries:

1. Routes generated for a center, including creator, creation date, and parameters.
2. Most frequently managed orders in final routes, classified by priority and zone.
3. Routes validated by a supervisor, including date and observations.
4. Driver performance on a route, grouped by priority and on-time delivery rate.
5. Comparison of routes from several centers by zone, priority, and criteria compliance.
6. Correlation between priority and performance by center; identification of the ten orders with the highest non-compliance rate, their coordinator and center; and comparison between drivers who requested rescheduling and the average on-time rate of their fleets.

2.5 Non-functional requirements

- **Performance and concurrency:** frequent queries about availability and deliveries must respond with low latency even when several coordinators are connected.
- **Availability:** driver and in-progress stop status must be reflected promptly during the workday.
- **Scalability and maintainability:** the architecture must be decoupled, documented, modular, and prepared for new functionality.
- **Cross-platform operation:** the web application must run in the environments expected by the team and users.

- **Development quality:** the project requires version control, planning, documented code, at least two design patterns, and unit tests for both backend and frontend.
- **Territorial network:** the connection network will be managed independently through a graph structure appropriate for dynamic traversal and searches.

3 Proposed technology stack and architecture

3.1 Technology stack

The solution will be implemented as a cross-platform web application organized into decoupled services. The selected technologies address the need for a maintainable interface, a strictly validated API, a transactional relational model, and a specialized component for the dynamic road network.

Layer	Technologies	Responsibility
Presentation	React, TypeScript, Vite, React Router	Web interface, navigation, forms, route queries, and report visualization.
State and data	TanStack Query	API consumption, caching, synchronization, and loading or error states.
Visualization	Recharts	Charts for priorities, compliance, performance, and center comparison.
Backend	Python, FastAPI, Pydantic	REST API, data validation, authentication, and use-case coordination.
Relational persistence	PostgreSQL, SQLAlchemy, Alembic	Users, orders, centers, routes, deliveries, rescheduling, and integrity constraints.
Graph persistence	Neo4j	Zones, roads, transit points, distances, estimated times, and alternative routes.
Testing	Pytest, Vitest, React Testing Library	Backend and frontend unit tests, including critical business rules.
Execution	Docker Compose	Reproducible environment for frontend, backend, PostgreSQL, and Neo4j.
Collaboration	GitHub, GitHub Projects, GitHub Actions	Version control, planning, review, and automated test execution.

3.2 Architecture and communication

The architecture will follow a layered approach. React will communicate with FastAPI through a REST API over HTTPS. FastAPI will separate HTTP routes, application services, repositories, and integrations with PostgreSQL and Neo4j. PostgreSQL will be the source of truth for transactional operations; Neo4j will be used to traverse the territorial network and suggest alternative routes. Communication between both stores will be coordinated by the service layer and not by the user interface.

- **Presentation layer:** typed React components with TypeScript, validated forms, and operational and reporting views.
- **API layer:** versioned REST endpoints, Pydantic schemas, and consistent error handling.
- **Application layer:** use cases for registering orders, generating routes, validating routes, recording deliveries, and resolving rescheduling requests.
- **Persistence layer:** separate repositories for PostgreSQL and Neo4j, with relational transactions and specialized graph queries.
- **Security layer:** access and refresh tokens, Argon2-protected passwords, and authorization by role and enabled center.

3.3 Patterns, quality, and operation

The project will apply the **Repository** pattern to isolate data access, **Service Layer** to centralize business rules, and **Strategy** to support new balancing criteria without modifying the general route-generation flow. Code will be organized by responsibility, with descriptive names, documentation for public functions, and validation at each service boundary.

Unit tests will cover at least role authorization, duplicate-order prevention, exclusive driver assignment, criteria-based generation, delivery recording, and rescheduling creation. The frontend will be tested with Vitest and React Testing Library; the backend will be tested with Pytest. GitHub Actions will run these tests automatically on relevant changes.

Configuration will be managed through environment variables and an example file. Docker Compose will provide a reproducible local environment. The repository must include dependency manifests, a ‘.gitignore‘ file, the services required for local execution, and visible planning in GitHub Projects with tasks, assignees, statuses, dates, and milestones.

4 Extended Relational Conceptual Model (ERCM)

4.1 Conceptual view

The ERCM is organized around the root entity **USER** and a total, disjoint ISA specialization into LOGISTICS_COORDINATOR, DRIVER, OPERATIONS_SUPERVISOR, and

ADMINISTRATOR. Territorial and operational entities are connected through relationships with explicit cardinalities. Attributes belonging to M:N relationships are represented by **ROUTE_STOP** and **ROUTE_CRITERION** in the relational schema.

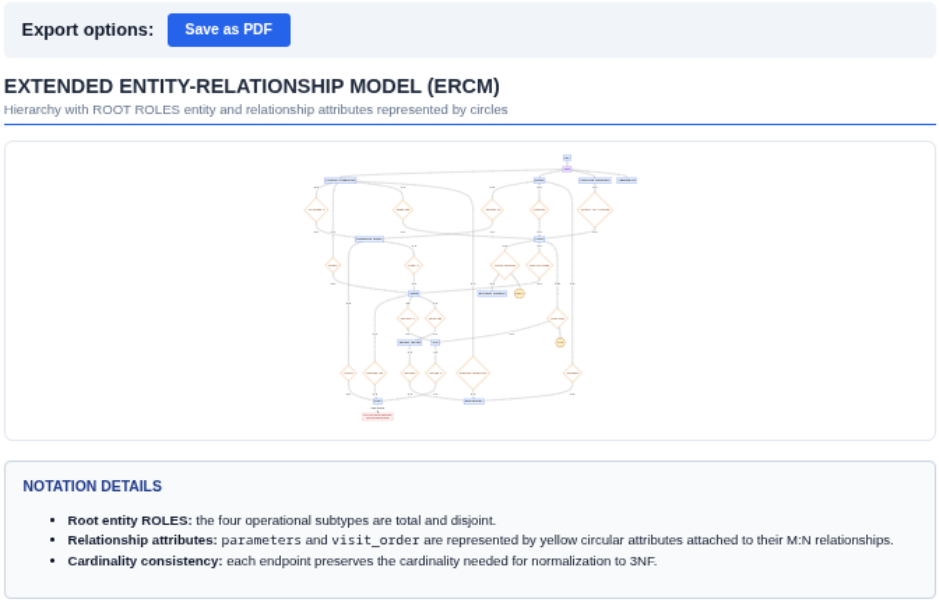


Figure 1: Extended Entity-Relationship Model (ERCM).

4.2 Main entities and relationships

- **Users and roles:** USER centralizes name, email, credentials, status, and session renewal. Each subtype adds its specific attributes.
- **Territory:** DISTRIBUTION_CENTER is related to ZONE through COVERS; a STOP is located in a zone. Road topology remains outside the relational model.
- **Load:** ORDER is entered by a coordinator, stored at a center, and delivered to a stop. A route contains the selected orders, resolving the connection trap between planning and execution.
- **Planning:** ROUTE is generated by a coordinator, originates at a center, may be validated by a supervisor, and is executed by a driver. ROUTE_STOP stores visit order and ROUTE_CRITERION stores the applied parameters.

- **Traceability:** each DELIVERY_RECORD documents an order; a failed delivery may originate a RESCHEDULING request submitted by a driver and resolved by a coordinator.

5 Integrity constraints

5.1 Entity integrity

Every table has a non-null and unique primary key. Primary keys include ‘id_user‘, ‘id_coordinator‘, ‘id_driver‘, ‘id_supervisor‘, ‘id_admin‘, ‘id_center‘, ‘id_zone‘, ‘id_stop‘, ‘id_order‘, ‘id_criterion‘, ‘id_route‘, ‘id_delivery‘, and ‘id_rescheduling‘. Associative tables use composite keys such as ‘(id_center,id_zone)‘, ‘(id_route,id_stop)‘, and ‘(id_route,id_criterion)‘.

5.2 Referential integrity

- Every user subtype references exactly one existing USER.
- ORDER references an existing coordinator, center, and stop; the route is nullable until the order is dispatched.
- STOP references an existing ZONE. CENTER_ZONE relates only existing centers and zones.
- ROUTE references the coordinator who generates it and its origin center; supervisor and driver may be null until those stages are completed.
- ROUTE_STOP and ROUTE_CRITERION may reference only existing routes, stops, and criteria.
- DELIVERY_RECORD references an existing ORDER and RESCHEDULING references an existing delivery record.

5.3 Business constraints

1. **No duplicates:** a unique constraint will be defined over the attributes that identify the same business order, together with application-level validation.
2. **Authorization:** a coordinator may register orders or generate routes only for centers present in COORDINATOR_CENTER.
3. **Exclusive assignment:** a driver may not have more than one incompatible active route; this will be guaranteed through transactional validation and a partial unique constraint where supported.
4. **Geographical consistency:** a stop must belong to a valid zone, and the center-zone combination must exist when orders or routes are assigned to a center.
5. **Criteria persistence:** parameters are stored as a snapshot in ROUTE_CRITERION, so later changes to a criterion do not alter historical traceability.

6. **Valid rescheduling:** only a registered and unsuccessful delivery may generate a request; status and resolution date must be consistent.

7. **Controlled states:** priority, delivery type, order status, route status, and delivery result must be restricted to system-defined domains.

6 Assessment of design correctness

6.1 Functional dependencies and normalization

From the attribute universe, the following representative functional dependencies are obtained:

$$\begin{aligned}
\texttt{id_user} &\rightarrow \text{general user data},\\
\texttt{id_order} &\rightarrow \text{order data, coordinator, center, and stop},\\
\texttt{id_stop} &\rightarrow \texttt{id_zone},\\
\texttt{id_route} &\rightarrow \text{dates, capacity, status, coordinator, and center},\\
(\texttt{id_route}, \texttt{id_stop}) &\rightarrow \texttt{visit_order},\\
(\texttt{id_route}, \texttt{id_criterion}) &\rightarrow \texttt{configuration_parameters}.
\end{aligned}$$

The dependency $\texttt{id_order} \rightarrow \texttt{id_zone}$ is not kept as an independent dependency because it follows transitively from $\texttt{id_order} \rightarrow \texttt{id_stop}$ and $\texttt{id_stop} \rightarrow \texttt{id_zone}$. Removing this redundancy avoids update anomalies.

Synthesis by determinant produces, among others, USER, the role subtypes, DISTRIBUTION_CENTER, ZONE, CENTER_ZONE, STOP, ORDER, BALANCING_CRITERION, ROUTE, ROUTE_STOP, ROUTE_CRITERION, COORDINATOR_CENTER, DRIVER_CENTER, DELIVERY_RECORD, and RESCHEDULING. Each relation contains attributes that depend on its key, on the whole key, and on no non-key attribute; therefore, the resulting schema satisfies 3NF under the declared dependencies.

6.2 Dependency preservation and lossless join

Dependencies are preserved because every determinant and its dependent attributes appear in a synthesized relation. Foreign keys preserve the semantic links and allow the information to be reconstructed through lossless joins. The direct ROUTE–ORDER association prevents losing the relationship between an expedition and its load when a stop contains orders from different routes.

6.3 Recommended implementation decisions

- Use transactions to generate routes, assign drivers, and register deliveries.
- Index foreign keys, operational statuses, dates, centers, and priorities to support reports.
- Apply concurrency control when checking availability and confirming assignments.

- Keep the road network in the graph engine and store in the relational model the references needed to connect stops and zones.
- Implement a decoupled layered or modular architecture, apply at least two patterns, and provide unit tests for backend and frontend.

7 Conclusions

This design aims to make the database reflect the real operation rather than merely present a list of tables. Roles represent different responsibilities; orders retain their origin, destination, and status; routes keep the criteria used to generate them; and delivery records make it possible to understand what happened at each stop. Information therefore remains traceable when a route changes, a delivery fails, or a rescheduling request is created.

ISA specialization prevents common user data from being repeated, while associative relationships represent routes, stops, and applied criteria correctly. The road network is kept in a graph engine because its connections and traversals change more often than administrative data. This separation keeps the relational model understandable and leaves room to search for alternatives when a delivery cannot be completed as planned.

Keys, references, and business constraints protect data consistency, while 3NF normalization reduces duplication and update anomalies. The proposed technology stack also provides a realistic path from this model to a maintainable application, with an accessible interface, a validated API, automated tests, and a reproducible environment. The result does not attempt to settle every implementation detail, but it offers a solid, understandable, and verifiable foundation for building the system.

Sources used

- International Council on Systems Engineering. *Requirements Engineering*, SEBoK v2.14. Available at: https://www.sebokwiki.org/wiki/Requirements_Engineering. Accessed October 9, 2026.
- University of Cape Town. *Chapter 6: Entity-Relationship Modelling*. Academic database material. Available at: https://www.cs.uct.ac.za/mit_notes/database/htmls/chp06.html. Accessed October 9, 2026.
- University of Cape Town. *Chapter 7: Enhanced Entity-Relationship Modelling*. Academic material on specialization, generalization, and ISA constraints. Available at: https://www.cs.uct.ac.za/mit_notes/database/htmls/chp07.html. Accessed October 9, 2026.
- University of Cape Town. *Chapter 8: Data Normalisation*. Academic material on functional dependencies and 1NF–3NF. Available at: https://www.cs.uct.ac.za/mit_notes/database/htmls/chp08.html. Accessed October 9, 2026.
- PostgreSQL Global Development Group. *Data Definition and Constraints*. Official documentation on keys, constraints, and referential integrity. Available at: https://www.postgresql.org/docs/current/ddl-constraints.html. Accessed October 9, 2026.

- Neo4j, Inc. *Get Started with Neo4j*. Official documentation on graph databases and Cypher. Available at: https://neo4j.com/docs/getting-started/current/. Accessed October 9, 2026.
- React. *React Documentation: Learn*. Official documentation on components and interface state. Available at: https://react.dev/learn. Accessed October 9, 2026.
- FastAPI. *FastAPI Documentation*. Official documentation on APIs, validation, and OpenAPI. Available at: https://fastapi.tiangolo.com/. Accessed October 9, 2026.
- Docker. *Get Started*. Official documentation on containers and reproducible environments. Available at: https://docs.docker.com/get-started/. Accessed October 9, 2026.
- GitHub. *About Projects*. Official documentation on planning with boards, milestones, and task tracking. Available at: https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects. Accessed October 9, 2026.

MATCOM